

Akademia Górniczo-Hutnicza im. Stanisława Staszica

Wydział Elektrotechniki, Automatyki, Informatyki i Inżynierii Biomedycznej

Projekt z przedmiotu
Obiektowe metody projektowania systemów

esp32-link

Aplikacja okienkowa na system GNU/Linux
do komunikacji z płytką ESP32

Autor: Paweł Michalcewicz

Kontakt: michalcewicz@student.agh.edu.pl

Prowadzący: prof. dr hab. inż. Bogusław Cyganek

Kraków 2026

Repozytorium: <https://github.com/mcvicz/esp32-link>

Historia zmian

Wersja	Data	Autor	Opis zmian
0.1	2026-06-12	P. Michalcewicz	Pierwsza wersja dokumentacji.

Spis treści

Lista oznaczeń i skrótów

1. Wstęp

2. Funkcjonalność (functionality)

2.1 Mikrokontroler (firmware)

2.2 Aplikacja desktopowa

3. Analiza problemu (problem analysis)

3.1 Współistnienie pętli zdarzeń Qt i asyncio

3.2 Reakcja wielu widgetów na te same zdarzenia

3.3 Reprezentacja stanu połączenia

3.4 Wymienność warstwy transportu

4. Projekt techniczny (technical design)

4.1 Architektura warstwowa

4.2 Diagramy UML

4.3 Zastosowane wzorce projektowe

4.3.1 Facade — Esp32Client

4.3.2 Strategy — Transport

4.3.3 State — ConnectionStateMachine

4.3.4 Command — ToggleLedCommand, PingCommand

4.3.5 Observer — sygnały Qt klienta

4.3.6 Singleton — Config.instance()

5. Opis realizacji (implementation report)

5.1 Protokół komunikacyjny

5.2 Maszyna stanów połączenia

5.3 Stałe sieciowe

5.4 Komentowanie kodu

6. Opis wykonanych testów (testing report)

6.1 Strategia testowania

6.2 Continuous Integration

6.3 Lista znalezionych i naprawionych błędów

7. Podręcznik użytkownika (user's manual)

7.1 Wymagania sprzętowe

7.2 Wymagania programowe

7.3 Uruchomienie pierwsze (runbook)

7.4 Diagnostyka problemów

8. Metodologia rozwoju i utrzymania systemu

8.1 Kontrola wersji

8.2 Automatyzacja jakości

8.3 Strategia ponownych prób połączenia

8.4 Procedura dodawania nowego polecenia

Bibliografia

Lista oznaczeń i skrótów

Skrót	Rozwinięcie
AP	Access Point — punkt dostępowy sieci Wi-Fi
API	Application Programming Interface — interfejs programistyczny
CI	Continuous Integration — ciągła integracja
ESP32	Rodzina mikrokontrolerów firmy Espressif Systems
FSM	Finite State Machine — automat skończony
GoF	Gang of Four — autorzy klasycznej książki o wzorcach projektowych
GUI	Graphical User Interface — graficzny interfejs użytkownika
JSON	JavaScript Object Notation — format wymiany danych
OOP	Object-Oriented Programming — programowanie obiektowe
RSSI	Received Signal Strength Indicator — wskaźnik siły sygnału
SSID	Service Set Identifier — nazwa sieci Wi-Fi
UML	Unified Modeling Language — ujednolicony język modelowania
UUID	Universally Unique Identifier — globalnie unikalny identyfikator
WSL	Windows Subsystem for Linux
WSL2	Druga generacja WSL z prawdziwym jądrem Linux
WSLg	Rozszerzenie WSL renderujące okna GUI Linuksa w Windows

1. Wstęp

Niniejszy dokument stanowi dokumentację projektu esp32-link — aplikacji okienkowej działającej na systemie GNU/Linux, której zadaniem jest komunikacja z mikrokontrolerem ESP32 za pośrednictwem bezprzewodowej sieci Wi-Fi. Projekt powstał jako zaliczenie kursu MPiMS prof. dr hab. inż. Bogusława Cyganka.

Założenia projektowe są celowo minimalne (jeden mikrokontroler, jedna aplikacja desktopowa). Nacisk położony jest na architekturę systemu oraz na zastosowanie wzorców projektowych: warstwowy podział kodu źródłowego, sześć udokumentowanych wzorców (Facade, Strategy, State, Command, Observer, Singleton), oraz pełna dokumentacja w formie tekstowej i diagramowej.

Kompletny kod źródłowy oraz materiały uzupełniające (siedem dokumentów Markdown, siedem diagramów PlantUML, skrypt budowy pliku PDF, testy jednostkowe i integracyjne, pipeline CI w GitHub Actions) znajdują się w publicznym repozytorium pod adresem:

<https://github.com/mcvicz/esp32-link>

Niniejszy dokument zawiera kompletny opis systemu na poziomie wymaganym do zrozumienia jego struktury i działania. Szczegóły implementacyjne, listingi kodu oraz dokumentacja techniczna każdego modułu są dostępne w repozytorium — odpowiednie pliki wskazywane są w treści dokumentu.

2. Funkcjonalność (functionality)

2.1 Mikrokontroler (firmware)

Firmware uruchamiany na płytce ESP32-WROOM-32 realizuje następujące funkcje:

- Uruchamia software'owy punkt dostępowy Wi-Fi o nazwie ESP32-Console (hasło: esp32pass).
- Obsługuje serwer WebSocket pod adresem ws://192.168.4.1:81/ws.
- Próbuje co 500 ms trzy wartości telemetryczne: wewnętrzną temperaturę układu (°C), ilość wolnej pamięci podręcznej heap (bajty), oraz wskaźnik siły sygnału RSSI (dBm) pierwszego podłączonego klienta.
- Rozsyła ramki telemetryczne do wszystkich podłączonych klientów w formacie JSON (jedna ramka na linię, zakończona znakiem nowej linii).
- Przyjmuje polecenia od klienta: toggle_led (przełączenie stanu diody LED na GPIO 2) oraz ping (odbicie testowe).
- Odpowiada na każde polecenie ramką potwierdzającą (ack) z identyfikatorem polecenia.
- Pomija rozsyłanie telemetryki gdy brak podłączonych klientów (oszczędność cykli CPU).
- Loguje do portu szeregowego (115200 baud) zdarzenia startu, połączeń klientów oraz wykonanych poleceń w formie czytelnej dla człowieka.

2.2 Aplikacja desktopowa

Aplikacja desktopowa uruchamiana na hoście z systemem GNU/Linux (lub WSL2 z WSLg w środowisku Windows) realizuje następujące funkcje:

- łączy się ze wskazanym serwerem WebSocket (domyślny adres pre-wypełniony w polu URL).
- Wizualizuje dane telemetryczne na trzech wykresach czasu rzeczywistego w przesuującym się oknie 60-sekundowym, po jednym wykresie na każdą wartość (temperatura, heap, RSSI).
- Umożliwia użytkownikowi wystanie poleceń (toggle_led, ping) za pomocą przycisków GUI; przyciski są dezaktywowane gdy brak aktywnego połączenia.
- Sygnalizuje aktualny stan połączenia (Disconnected, Connecting, Connected, Reconnecting, Error) kolorowanym wskaźnikiem w pasku statusu wraz z opcjonalnym komunikatem błędu.
- Automatycznie próbuje ponownie nawiązać połączenie po jego utracie z wykorzystaniem strategii wykładniczego backoff (1 s, 2 s, 4 s, 8 s, a następnie 8 s do momentu przerwania przez użytkownika).
- Zapamiętuje ostatnio użyty adres URL oraz geometrię okna pomiędzy uruchomieniami (plik konfiguracyjny ~/.config/esp32-link/config.json).

Pełną specyfikację funkcjonalną oraz listę wymagań niefunkcjonalnych zawiera plik *docs/01-requirements.md* w repozytorium.

3. Analiza problemu (problem analysis)

Komunikacja z mikrokontrolerem przez Wi-Fi jest stosunkowo standardowym zagadnieniem, natomiast projekt postawiony na aplikacji okienkowej w języku Python wymusza rozwiązanie kilku konkretnych problemów architektonicznych:

3.1 Współistnienie pętli zdarzeń Qt i asyncio

Biblioteka PySide6 (Qt) opiera się na własnej synchronicznej pętli zdarzeń, natomiast biblioteka websockets działa w środowisku asyncio. Naiwne połączenie obu pętli (np. await w wątku Qt) prowadzi do zawieszenia interfejsu użytkownika podczas operacji sieciowych. Rozwiązanie: pętla asyncio jest uruchamiana w osobnym wątku tła, a komunikacja z wątkiem GUI odbywa się wyłącznie poprzez sygnały Qt z domyślnym połączeniem typu Qt::QueuedConnection (które jest bezpieczne wątkowo).

3.2 Reakcja wielu widgetów na te same zdarzenia

Wykres telemetryczny, panel sterowania oraz pasek statusu muszą reagować na te same zdarzenia (nowa próbka telemetryczna, zmiana stanu połączenia, błąd transportu). Wprost-zorientowane podejście — wzajemne odwołania między widgetami — szybko prowadzi do silnego sprzężenia. Rozwiązaniem jest wzorzec Obserwatora w jego idiomatycznej formie Qt (sygnały i sloty).

3.3 Reprezentacja stanu połączenia

Połączenie z mikrokontrolerem może znajdować się w jednym z pięciu wyraźnie rozróżnialnych stanów (rozłączony, łączenie, połączony, ponowne łączenie, błąd). Pierwsza iteracja projektu modelowała stan dwiema zmiennymi logicznymi, co prowadziło do błędów typu „interfejs pokazuje rozłączenie, ale klient w tle dalej próbuje się łączyć”. Konieczne było wprowadzenie jawnego, walidowanego automatu stanu (Finite State Machine).

3.4 Wymienność warstwy transportu

Choć projekt aktualnie wykorzystuje wyłącznie transport WebSocket, w przyszłości może zaistnieć potrzeba dodania transportu szeregowego (USB) — w szczególności, gdy z powodów technicznych Wi-Fi nie jest dostępne. Architektura musi umożliwić podmianę warstwy transportowej bez modyfikacji warstwy aplikacyjnej ani warstwy GUI.

Każdy z wymienionych problemów został rozwiązany za pomocą odpowiednio dobranego wzorca projektowego. Szczegóły w sekcji 4.

4. Projekt techniczny (technical design)

4.1 Architektura warstwowa

Aplikacja desktopowa podzielona jest na cztery warstwy, ze ścisłym jednokierunkowym kierunkiem zależności od góry do dołu. Górne warstwy mogą zależeć od dolnych, ale nigdy odwrotnie.

ui/	widgety PySide6, sygnały i sloty Qt
application/	fasada klienta (Esp32Client), obiekty poleceń (Command)
domain/	modele danych, automat stanu (czysta logika; brak Qt, brak sieci)
infrastructure/	abstrakcja transportu, codec JSON, transport WebSocket

Warstwa domain/ nie importuje niczego z biblioteki Qt ani z biblioteki websockets, co czyni ją w pełni testowalną bez uruchamiania pętli zdarzeń ani otwierania gniazda sieciowego. Testy warstwy domain/ wykonują się w czasie poniżej kilkudziesięciu milisekund.

4.2 Diagramy UML

Pełen zestaw diagramów UML znajduje się w folderze docs/uml/ repozytorium w postaci plików źródłowych PlantUML:

Plik	Typ diagramu UML	Zawartość
use_case.puml	Diagram przypadków użycia	Aktorzy (użytkownik, ESP32) oraz przypadki użycia z relacjami includes/extends.
class_desktop.puml	Diagram klas (+ pakietów)	Cztery warstwy aplikacji desktopowej, wszystkie sześć wzorców oznaczonych stereotypami.

class_firmware.puml	Diagram klas	Komponenty logiczne firmware'u (cfg, Telemetry, Protocol, CommandDispatcher, WsServer).
sequence_connect.puml	Diagram sekwencji	Ścieżka szczęśliwa procesu nawiązywania połączenia.
sequence_telemetry.puml	Diagram sekwencji	Pętla telemetryczna 2 Hz oraz round-trip polecenia.
state_connection.puml	Diagram maszyny stanów	Pełen automat ConnectionState z przejściami.
deployment.puml	Diagram wdrożenia	Host Linux + ESP32 połączone przez Wi-Fi soft AP.

Diagramy są walidowane podczas rozwoju projektu poprzez automatyczne renderowanie do formatu SVG za pomocą serwisu plantuml.com — gwarantuje to ich aktualność oraz poprawność składniową.

4.3 Zastosowane wzorce projektowe

W projekcie zastosowano sześć wzorców z klasycznej książki Gang of Four. Każdy z wzorców został wybrany jako odpowiedź na konkretny problem opisany w sekcji 3, a nie jako element ozdobny mający spełnić wymagania kursowe.

4.3.1 Facade — Esp32Client

Klasa `application/client.py::Esp32Client` udostępnia warstwie GUI minimalny interfejs (metody `connect`, `disconnect`, `send` oraz cztery sygnały Qt) ukrywając wewnętrznie cały stack: warstwę transportu, codec, automat stanu oraz logikę ponownego łączenia. GUI nie importuje bezpośrednio bibliotek `websockets` ani `asyncio`.

4.3.2 Strategy — Transport

Abstrakcyjna klasa `infrastructure/transport.py::Transport` definiuje interfejs transportu tekstowego (metody `connect`, `disconnect`, `send` oraz `async iterator messages`). Aktualna jedyna konkretna implementacja to `WebSocketTransport`. Wprowadzenie abstrakcji pozwala na podmianę transportu (np. na `SerialTransport`) bez modyfikacji warstwy aplikacyjnej.

4.3.3 State — ConnectionStateMachine

Wyliczeniowy typ `ConnectionState` definiuje pięć stanów połączenia. Klasa `ConnectionStateMachine` (warstwa domain/) przechowuje tablicę dozwolonych przejść i wymusza ich poprawność rzucając `IllegalTransitionError` przy próbie niedozwolonego przejścia.

4.3.4 Command — ToggleLedCommand, PingCommand

Każde polecenie wysyłane do mikrokontrolera reprezentowane jest jako obiekt typu `Command` (dataclass z polem `cmd_id` typu `UUID v4` generowanym automatycznie). Metoda `serialize()`

przekształca obiekt w słownik JSON zgodny z protokołem. Dodanie nowego polecenia nie wymaga zmian w klasie Esp32Client — wystarczy nowa podklasa Command.

4.3.5 Observer — sygnały Qt klienta

Esp32Client jako QObject udostępnia cztery sygnały Qt: telemetry_received, ack_received, state_changed, error_occurred. Każdy widget GUI podpiną swój slot do interesującego go sygnału. Jest to wzorzec Observer w idiomatycznej formie biblioteki Qt — sygnały automatycznie dostarczane są pomiędzy wątkami w sposób bezpieczny.

4.3.6 Singleton — Config.instance()

Klasa Config przechowuje ustawienia użytkownika (ostatnio użyty adres URL, geometria okna), zapisywane do pliku ~/.config/esp32-link/config.json. Dostęp wyłącznie poprzez metodę klasową Config.instance() z synchronizacją wątkową (threading.Lock). Wybór wzorca Singleton był świadomym kompromisem — jego zaletą jest jednolity punkt dostępu do konfiguracji z dowolnego miejsca w kodzie.

Szczegółowy opis każdego z wzorców (problem, alternatywy rozważane, kompromisy) wraz z fragmentami kodu znajduje się w pliku *docs/03-design.md* w repozytorium. Jest to najobszerniejszy z dokumentów towarzyszących projektowi.

5. Opis realizacji (implementation report)

5.1 Protokół komunikacyjny

Komunikacja pomiędzy aplikacją desktopową a mikrokontrolerem odbywa się za pomocą pojedynczego połączenia WebSocket. Wszystkie komunikaty są obiektami JSON przesyłanymi jako ramki tekstowe WebSocket, jeden obiekt na ramkę, zakończony znakiem nowej linii. Zdefiniowano trzy typy wiadomości:

Telemetria (ESP32 → klient):

```
{"type": "telemetry", "ts": 12345, "temp_c": 24.5, "free_heap": 235200, "rssi": -42}\n
```

Potwierdzenie polecenia (ESP32 → klient):

```
{"type": "ack", "cmd_id": "<uuid>", "ok": true, "msg": "led on"}\n
```

Polecenie (klient → ESP32):

```
{"type": "cmd", "cmd_id": "<uuid>", "action": "toggle_led"}\n
```

Pełna specyfikacja każdego pola, lista wszystkich zdefiniowanych poleceń, oraz polityka obsługi błędów (nieparsowalny JSON, nieznaný typ wiadomości, brakujące pola) znajdują się w pliku *docs/04-protocol.md*.

5.2 Maszyna stanów połączenia

Tabela dozwolonych przejść stanu ConnectionState:

Stan początkowy	Dozwolone stany docelowe
DISCONNECTED	CONNECTING
CONNECTING	CONNECTED, ERROR, DISCONNECTED
CONNECTED	DISCONNECTED, RECONNECTING, ERROR
RECONNECTING	CONNECTED, ERROR, DISCONNECTED
ERROR	DISCONNECTED, CONNECTING, RECONNECTING

Pełny diagram automatu zawarty jest w pliku *docs/uml/state_connection.puml*, a opis przejść oraz miejsca ich wywołania w kodzie — w pliku *docs/05-state-machine.md*.

5.3 Stałe sieciowe

Parametr	Wartość
SSID sieci Wi-Fi	ESP32-Console
Hasło sieci Wi-Fi (WPA2)	esp32pass
Adres IP mikrokontrolera (soft AP)	192.168.4.1
Port serwera WebSocket	81
Ścieżka serwera WebSocket	/ws
Częstotliwość rozsyłania telemetrii	2 Hz (co 500 ms)
Strategia ponownych prób (backoff)	1 s, 2 s, 4 s, 8 s, dalej 8 s

5.4 Komentowanie kodu

W projekcie zastosowano zasadę „kod powinien dokumentować się sam” — komentarze stosowane są jedynie w miejscach, gdzie powód niejednoznaczny lub gdy wyjaśnienie odnosi się do tła historycznego decyzji (przykład: komentarz w pliku *firmware/src/main.cpp* przy wywołaniu *temperatureRead()* wyjaśniający usunięcie historycznej konwersji jednostek z Fahrenheita na Celsjusza). Pełna dokumentacja sygnatur typów odbywa się poprzez kompletny system adnotacji typów Pythona (*type hints*) oraz typowanych *dataclasses*.

6. Opis wykonanych testów (testing report)

6.1 Strategia testowania

Pakiet testowy (folder *desktop/tests/*) zawiera 38 testów podzielonych na pięć plików, mapujących się jednoznacznie na cztery warstwy architektury aplikacji desktopowej:

Plik testowy	Warstwa	Pokrycie
<i>test_codec.py</i>	infrastructure	10 testów: enkodowanie/dekodowanie JSON, błędne wejście, brakujące pola, puste linie.
<i>test_state_machine.py</i>	domain	7 testów: dozwolone

		przejścia, niedozwolone przejścia, ścieżka szczęśliwa i awaryjna.
test_commands.py	application	5 testów: serializacja zgodna z protokołem, unikalność UUID, jawne nadpisanie cmd_id.
test_client_integration.py	application + infrastructure	5 testów: pełen klient przeciwko atrapie serwera WebSocket (fake_ws_server).
test_ui.py	ui	11 testów: konstrukcja widgetów, propagacja sygnałów, włączanie/wyłączanie przycisków w zależności od stanu.

Atrapa serwera (fixture fake_ws_server) jest w istocie pełnoprawnym serwerem WebSocket z biblioteki websockets, uruchamianym na losowym wolnym porcie w wątku tła. Pozwala to testować klienta przeciwko prawdziwemu kanałowi WebSocket bez konieczności podłączania fizycznej płytki ESP32. Suma wszystkich testów wykonuje się w czasie poniżej 7 sekund.

6.2 Continuous Integration

Repozytorium zawiera workflow GitHub Actions (.github/workflows/ci.yml) z dwoma równolegle wykonywanymi zadaniami:

- python-tests — instaluje zależności runtime'u Qt (libegl1, libgl1, libxkbcommon-x11-0 i pozostałe), instaluje uv, wykonuje uv sync oraz uv run pytest w trybie offscreen (QT_QPA_PLATFORM=offscreen).
- firmware-build — instaluje PlatformIO i wykonuje pio run w folderze firmware/ celem weryfikacji, że kod C++ kompiluje się czysto przeciwko zafiksowanej wersji frameworka.

Oba zadania uruchamiane są automatycznie przy każdym pushu i pull requestie na branchu master. Wszystkie commity widoczne w repozytorium przeszły kompletny pipeline CI.

6.3 Lista znalezionych i naprawionych błędów

W trakcie wytwarzania projektu zidentyfikowano i naprawiono następujące istotne błędy:

#	Opis błędu	Objaw	Rozwiązanie (commit)
1	Niepoprawna konwersja jednostek w firmware. Pierwsza wersja stosowała warunkową konwersję °F → °C opartą o makro ESP_ARDUINO_VERSION_MAJOR.	Wykres temperatury pokazywał −3 °C w temperaturze pokojowej 23 °C.	Usunięcie konwersji (commit 5871e9f).

	Konwersja była poprawna dla Arduino-ESP32 < 2.0.14, ale błędna dla nowszych wersji, które zwracają już °C.		
2	Brak dozwolonego przejścia ERROR → RECONNECTING w tabeli przejść automatu stanu. Klient w tle podejmował kolejne próby łączenia, ale FSM blokował tę zmianę stanu.	Pasek statusu pozostawał czerwony (Error) mimo że klient nadal próbował łączyć się w nieskończoność.	Dodanie przejścia ERROR → RECONNECTING (commit 9607ac5).
3	Niewidoczność karty Wi-Fi z poziomu WSL2. W trybie domyślnym (NAT) WSL2 nie eksponuje karty Wi-Fi — pakiety do 192.168.4.1 są kierowane przez kartę Ethernet i kończą się komunikatem Network is unreachable.	ping 192.168.4.1 z poziomu WSL nie działa, ten sam ping z Windowsa działa.	Włączenie trybu networkingMode=mirrored oraz firewall=false w pliku .wslconfig. Rozwiązanie opisane w docs/07-build-and-run.md.

Pełna lista testów oraz instrukcja ich uruchomienia znajdują się w pliku *docs/06-testing.md*.

7. Podręcznik użytkownika (user's manual)

7.1 Wymagania sprzętowe

- Płytką rozwojową ESP32-WROOM-32 z konwerterem USB-UART (np. CP2102 lub CH340).
- Kabel USB obsługujący transmisję danych (nie tylko ładowanie).
- Komputer z kartą Wi-Fi 2.4 GHz — ESP32 nie obsługuje pasma 5 GHz.

7.2 Wymagania programowe

- System operacyjny Linux (testowano na Ubuntu 24.04 oraz WSL2 z WSLg w środowisku Windows 11).
- Python 3.12 lub nowszy.
- Menedżer środowiska Python uv (instalacja: `curl -Lsf https://astral.sh/uv/install.sh | sh`).
- PlatformIO (dla operacji na firmware) — zalecana instalacja po stronie Windowsa, ponieważ system WSL2 domyślnie nie udostępnia urządzeń USB.

7.3 Uruchomienie pierwsze (runbook)

Pełny przewodnik instalacji i uruchomienia, zawierający również tabelę rozwiązywania problemów (8 najczęstszych objawów z przyczynami i sposobem naprawy), znajduje się w pliku README.md w głównym folderze repozytorium. W skrócie:

Krok 1 — Firmware (terminal Windows PowerShell):

```
cd C:\...\esp32-link\firmware
python -m platformio run --target upload      # kompilacja i wgranie firmware'u
python -m platformio device monitor          # monitor portu szeregowego 115200 baud
```

Oczekiwany komunikat startowy: [boot] esp32-link firmware booted, [wifi] AP started: ESP32-Console @ 192.168.4.1, [ws] listening on port 81 path /ws.

Krok 2 — Sieć Wi-Fi:

Połączyć komputer z siecią o SSID „ESP32-Console” i hasło „esp32pass”. System Windows wyświetli ostrzeżenie „brak internetu” — jest to oczekiwane, ESP32 nie pełni roli routera.

Krok 3 — Aplikacja desktopowa (terminal WSL lub Linux):

```
cd ../esp32-link/desktop
uv sync          # jednorazowa instalacja zależności
uv run esp32-link # uruchomienie GUI
```

Po kliknięciu przycisku Connect wskaźnik statusu zmienia kolor z szarego (Disconnected) na zielony (Connected), a trzy wykresy zaczynają zapełniać się danymi w częstotliwości 2 Hz. Przyciski Toggle LED oraz Ping stają się aktywne w stanie Connected.

7.4 Diagnostyka problemów

Tabela rozwiązywania najczęstszych problemów (9 typowych objawów wraz z opisem przyczyny i sposobem naprawy) znajduje się w pliku *docs/07-build-and-run.md* w sekcji Troubleshooting. Obejmuje m.in.: brak komendy pio, brak portu USB w WSL, niewidoczność sieci Wi-Fi, błędy konfiguracji WSL2 oraz brakujące biblioteki Qt.

8. Metodologia rozwoju i utrzymania systemu

8.1 Kontrola wersji

Projekt prowadzony jest w systemie kontroli wersji Git. Repozytorium publiczne dostępne pod adresem: <https://github.com/mcvicz/esp32-link>

. Każda istotna zmiana wprowadzana jest jako pojedynczy commit z opisowym komunikatem; nie stosuje się commitów łączących wiele niezwiązanych zmian. Historia commitów odzwierciedla kolejność prac (scaffold → firmware → core klienta → UI → testy → dokumentacja).

8.2 Automatyzacja jakości

- Linting i formatowanie: ruff check oraz ruff format z konfiguracją zafiksowaną w pyproject.toml (zestaw reguł E, F, I, UP, B; długość linii 100 znaków; target Python 3.12).
- Testy: uv run pytest — 38 testów wykonujących się w czasie poniżej 7 sekund.
- Continuous Integration: GitHub Actions, dwa zadania (python-tests, firmware-build) uruchamiane przy każdym pushu i pull requestie.

- Dokumentacja: każda zmiana w protokole, architekturze lub liście wzorców musi być odzwierciedlona w odpowiednim pliku Markdown w folderze docs/.

8.3 Strategia ponownych prób połączenia

Zaimplementowano backoff wykładniczy: kolejne próby ponownego łączenia odbywają się po 1, 2, 4 i 8 sekundach, a następnie co 8 sekund w nieskończoność do momentu przerwania przez użytkownika. Użytkownik może w dowolnym momencie przerwać ponawianie klikając przycisk Disconnect — przerwanie odbywa się „czysto” poprzez sygnał `asyncio.Event` obserwowany przez pętlę `asyncio` w wątku tła.

8.4 Procedura dodawania nowego polecenia

Aby dodać nowe polecenie do protokołu (np. restart restartujący mikrokontroler), należy:

1. Dodać nową klasę dziedziczącą po `Command` w pliku `desktop/src/esp32_link/application/commands.py` (wystarczy zaimplementować metodę `action()`).
2. Dodać przycisk w pliku `desktop/src/esp32_link/ui/control_panel.py` oraz podpiąć sygnał do slotu w `main_window.py`.
3. Dodać obsługę w `firmware/src/main.cpp::CommandDispatcher::dispatch` (gałąź `else if`).
4. Zaktualizować specyfikację protokołu w `docs/04-protocol.md` (sekcja „Defined actions”).
5. Dodać test serializacji w `desktop/tests/test_commands.py`.
6. Przeflash firmware oraz uruchomić `uv run pytest` celem weryfikacji.

Bibliografia

- [1] E. Gamma, R. Helm, R. Johnson, J. Vlissides. „Design Patterns: Elements of Reusable Object-Oriented Software”. Addison-Wesley, 1994.
- [2] Dokumentacja Arduino-ESP32: <https://docs.espressif.com/projects/arduino-esp32/>
- [3] Dokumentacja PySide6 (Qt for Python): <https://doc.qt.io/qtforpython-6/>
- [4] Dokumentacja biblioteki websockets (Python): <https://websockets.readthedocs.io/>
- [5] Specyfikacja protokołu WebSocket: RFC 6455, <https://datatracker.ietf.org/doc/html/rfc6455>
- [6] Dokumentacja biblioteki `ESPAsyncWebServer`: <https://github.com/ESP32Async/ESPAsyncWebServer>
- [7] Dokumentacja biblioteki `ArduinoJson v7`: <https://arduinojson.org/v7/>
- [8] Dokumentacja `PlatformIO`: <https://docs.platformio.org/>
- [9] Dokumentacja narzędzia `uv` (Astral): <https://docs.astral.sh/uv/>
- [10] Dokumentacja narzędzia `ruff` (Astral): <https://docs.astral.sh/ruff/>

[11] Dokumentacja PlantUML: <https://plantuml.com/>